

The `resources` parameter provides agents with access to local files that serve as contextual knowledge, functioning like a knowledge base that informs the agent's responses. It accepts an array of file paths that must start with the `file://` prefix, supporting specific files (`file://README.md`), glob patterns for multiple files (`file://docs/**/*.md`), and both absolute and relative paths.

These resources are loaded into the agent's context at startup, allowing it to reference project documentation, coding standards, or any relevant files when responding to queries, and work alongside the `prompt` parameter to create well-informed, project-aware agents that understand specific codebases, requirements, or organizational knowledge.

Objective

Learn how to use the `resources` parameter to provide agents with project-specific knowledge.

Instructions

Step 1: Create Additional Context Files

1. Create coding standard documentation

```
1 cat > docs/coding-standards.md << 'EOF'
2 # Coding Standards
3
4 ## JavaScript Style Guide
5 - Use ES6+ features
6 - Prefer const over let
7 - Use arrow functions for callbacks
8 - Always use semicolons
9
10 ## React Guidelines
11 - Use functional components with hooks
12 - Keep components small and focused
13 - Use TypeScript for type safety
14 EOF
```

2. Create project structure documentation

```
1 cat > docs/project-structure.md << 'EOF'
2 # Project Structure
3
4 ---
5 src/
6 |--- components/ # Reusable UI components
7 |--- pages/      # Page-level components
8 |--- hooks/      # Custom React hooks
9 |--- utils/      # Utility functions
10 |--- services/   # API and external services
11 ---
12 EOF
```

3. Create API Guideline

```
1 cat > docs/api-guidelines.md << 'EOF'
2 # API Guidelines
3
4 ## REST API Standards
5 - Use RESTful endpoints with proper HTTP methods
6 - Implement consistent error handling with status codes
7 - Use JSON for request/response payloads
8 - Include API versioning in URLs (/api/v1/)
```

```
9
10 ## Authentication
11 - Use JWT tokens for stateless authentication
12 - Implement refresh token rotation
13 - Secure sensitive endpoints with proper middleware
14
15 ## Data Validation
16 - Validate all input data on server side
17 - Use schema validation libraries (Joi, Yup)
18 - Return descriptive error messages for validation failures
19 EOF
```

4. Create testing strategy

```
1 cat > docs/testing-strategy.md << 'EOF'
2 # Testing Strategy
3
4 ## Unit Testing
5 - Test individual functions and components in isolation
6 - Use Jest for JavaScript testing framework
7 - Aim for 80%+ code coverage on critical business logic
8 - Mock external dependencies and API calls
9
10 ## Integration Testing
11 - Test component interactions and data flow
12 - Use React Testing Library for component testing
13 - Test API endpoints with supertest
14 - Validate database operations with test fixtures
15
16 ## End-to-End Testing
17 - Use Cypress for full user workflow testing
18 - Test critical user journeys and business processes
19 - Run E2E tests in CI/CD pipeline before deployment
20 EOF
```

5. Create deployment guideline

```
1 cat > docs/deployment-guide.md << 'EOF'
2 # Deployment Guide
3
4 ## Environment Setup
5 - **Development**: Local development with hot reload
6 - **Staging**: Pre-production testing environment
7 - **Production**: Live application with monitoring
8
9 ## CI/CD Pipeline
10 1. Code commit triggers automated tests
11 2. Build and package application artifacts
12 3. Deploy to staging for integration testing
13 4. Manual approval gate for production deployment
14 5. Blue-green deployment with rollback capability
15
16 ## Monitoring & Logging
17 - Application performance monitoring with New Relic
18 - Error tracking with Sentry
19 - Structured logging with Winston
20 - Health check endpoints for load balancer
21 EOF
```

6. Create security checklist

```
1 cat > docs/security-checklist.md << 'EOF'
2 # Security Checklist
3
4 ## Input Validation
5 - [ ] Sanitize all user inputs
6 - [ ] Validate data types and formats
7 - [ ] Implement rate limiting on API endpoints
8 - [ ] Use parameterized queries to prevent SQL injection
9
10 ## Authentication & Authorization
11 - [ ] Implement secure password policies
12 - [ ] Use HTTPS for all communications
13 - [ ] Implement proper session management
14 - [ ] Apply principle of least privilege for user roles
15
16 ## Data Protection
17 - [ ] Encrypt sensitive data at rest
18 - [ ] Use secure communication protocols (TLS 1.3)
19 - [ ] Implement proper backup and recovery procedures
20 - [ ] Regular security audits and penetration testing
21 EOF
```

Step 2: Create Agent with Context Resources

```
1 cat > .kiro/agents/project-expert.json << 'EOF'
2 {
3   "name": "project-expert",
4   "description": "Project expert with comprehensive context knowledge",
5   "prompt": "You are a project expert with deep knowledge of this specific codebase. You understand the project structure, coding standards, and architectural decisions. U
6   "resources": [
7     "file://README.md",
8     "file://docs/coding-standards.md",
9     "file://docs/project-structure.md",
10    "file://src/app.js"
11  ]
12 }
13 EOF
```

Step 3: Create Agent with Dynamic Resources

```
1 cat > .kiro/agents/dynamic-context.json << 'EOF'
2 {
3   "name": "dynamic-context",
4   "description": "Agent with dynamic context patterns",
5   "prompt": "You are a development assistant with access to all project documentation and source code. Say 'I have no information.' if it cannot be found in context.",
6   "resources": [
7     "file://README.md",
8     "file://docs/**/*.md",
9     "file://src/**/*.js"
10  ]
11 }
12 EOF
```

Step 4: Test Context Integration

Try using prompts from one agent configuration with another agent to observe the variations in their responses.

1. Test Project Expert:

```
1 kiro-cli chat --agent project-expert
```

List context files:

```
1 /context show
```

Expected:

```
[project-expert] > /context show

👤 Agent (project-expert):
  README.md (1 match)
  docs/coding-standards.md (1 match)
  docs/project-structure.md (1 match)
  src/app.js (1 match)

💜 Session (temporary):
  <none>

4 matched files in use:
👤 /Workshop/amazon-q-agent-lab/src/app.js (~10 tkns)
👤 /Workshop/amazon-q-agent-lab/README.md (~10 tkns)
👤 /Workshop/amazon-q-agent-lab/docs/project-structure.md (~70 tkns)
👤 /Workshop/amazon-q-agent-lab/docs/coding-standards.md (~140 tkns)

Total: ~160 tokens
```

Try these commands:

```
"What are the coding standards for this project?"
"Explain the project structure"
"Create a new component following project guidelines"
"What's in the main application file?"
```

2. Test Dynamic Context:

```
1 kiro-cli chat --agent dynamic-context
```

List context files:

```
1 /context show
```

Expected:

```
[dynamic-context] > /context show

👤 Agent (dynamic-context):
  README.md (1 match)
  docs/**/*.md (7 matches)
  src/**/*.js (1 match)

💜 Session (temporary):
  <none>

9 matched files in use:
👤 /Workshop/amazon-q-agent-lab/docs/deployment-guide.md (~160 tkns)
👤 /Workshop/amazon-q-agent-lab/README.md (~10 tkns)
👤 /Workshop/amazon-q-agent-lab/docs/coding-standards.md (~70 tkns)
👤 /Workshop/amazon-q-agent-lab/docs/api-guidelines.md (~140 tkns)
👤 /Workshop/amazon-q-agent-lab/docs/architecture.md (~10 tkns)
👤 /Workshop/amazon-q-agent-lab/docs/project-structure.md (~70 tkns)
👤 /Workshop/amazon-q-agent-lab/docs/security-checklist.md (~160 tkns)
👤 /Workshop/amazon-q-agent-lab/docs/testing-strategy.md (~150 tkns)
👤 /Workshop/amazon-q-agent-lab/src/app.js (~10 tkns)

Total: ~780 tokens
```

Try these commands:

```
"What documentation files are available in this project?"
"Summarize all the markdown files in the docs directory"
"What are our API guidelines and testing strategies?"
"Tell me about our deployment and security practices"
"What JavaScript files exist in the project?"
"Compare our coding standards with our security checklist"
```

Expected Response Differences:

- project-expert:** Limited to 4 specific files (README.md, coding-standards.md, project-structure.md, src/app.js), provides focused but limited context, cannot access additional documentation files
- dynamic-context:** Can discover and access ALL .md files via `docs/**/*.md` pattern (coding-standards.md, project-structure.md, api-guidelines.md, testing-strategy.md, deployment-guide.md, security-checklist.md), provides comprehensive project knowledge, dynamically finds new files that match patterns, can cross-reference multiple documentation sources

Verification

- ✔ Project expert references only specific predefined documentation (4 files)
- ✔ Dynamic context agent can access ALL documentation files via patterns (6+ files)
- ✔ Dynamic context provides more comprehensive project knowledge
- ✔ Dynamic context can cross-reference multiple documentation sources
- ✔ File patterns (`docs/**/*.md`) discover all markdown files automatically
- ✔ Project expert has focused but limited context scope
- ✔ Dynamic context demonstrates the power of wildcard resource patterns

Key Learnings

- `resources` parameter provides agents with file-based context
- File paths use `file://` prefix for local files
- Wildcard patterns** (`/**/*.md`) enable dynamic resource loading and comprehensive context
- Static resource lists provide focused but limited context
- Dynamic patterns scale automatically as documentation grows
- Context resources significantly improve response accuracy and scope